

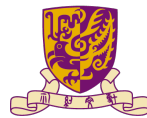
Network Programming

Parallelism basics and Collective Communication

Minchen Yu

SDS@CUHK-SZ

Spring 2026

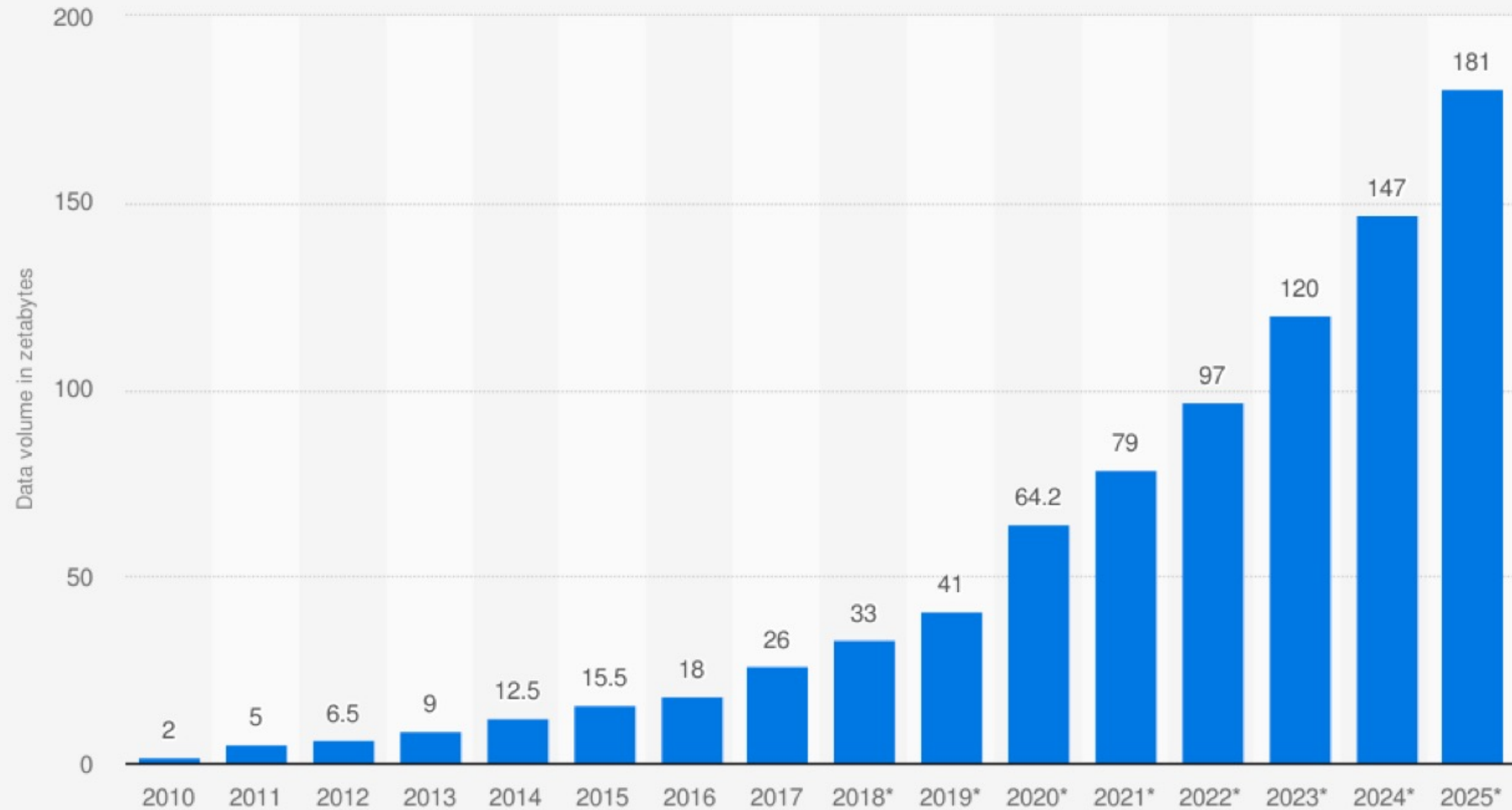


香港中文大學(深圳)
The Chinese University of Hong Kong, Shenzhen



SCHOOL OF
DATA SCIENCE
數據科學學院

Volume of data/information created, captured, copied, and consumed worldwide from 2010 to 2025 (in zettabytes)



Sources

IDC; Seagate; Statista estimates
© Statista 2021

Additional Information:

Worldwide; 2010 to 2020



How do we
compute?

Recap: Scale-out

Lots of cheap, commodity PCs, each with disk and CPU

- 100s to 1000s of PCs in cluster

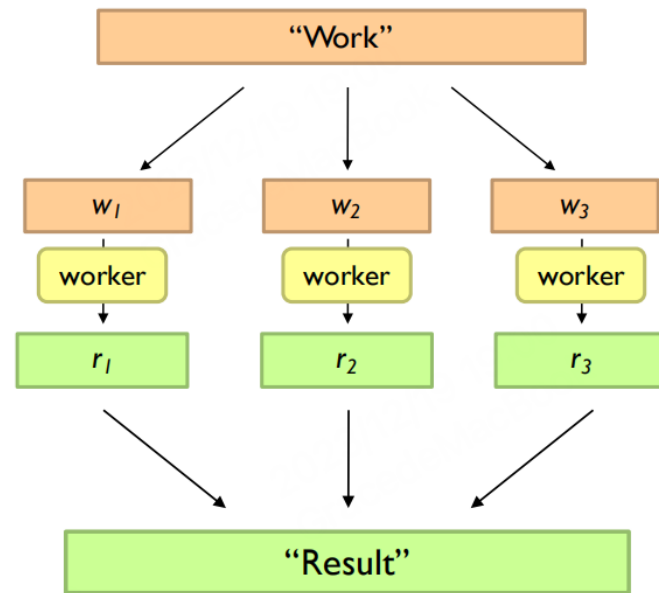
High aggregate compute and storage capacity

Spread computation across many machines

- High compute capacity and I/O bandwidth
- Parallelize data processing

Parallel data processing

Basic idea: Split up workload across machines/workers



Partition

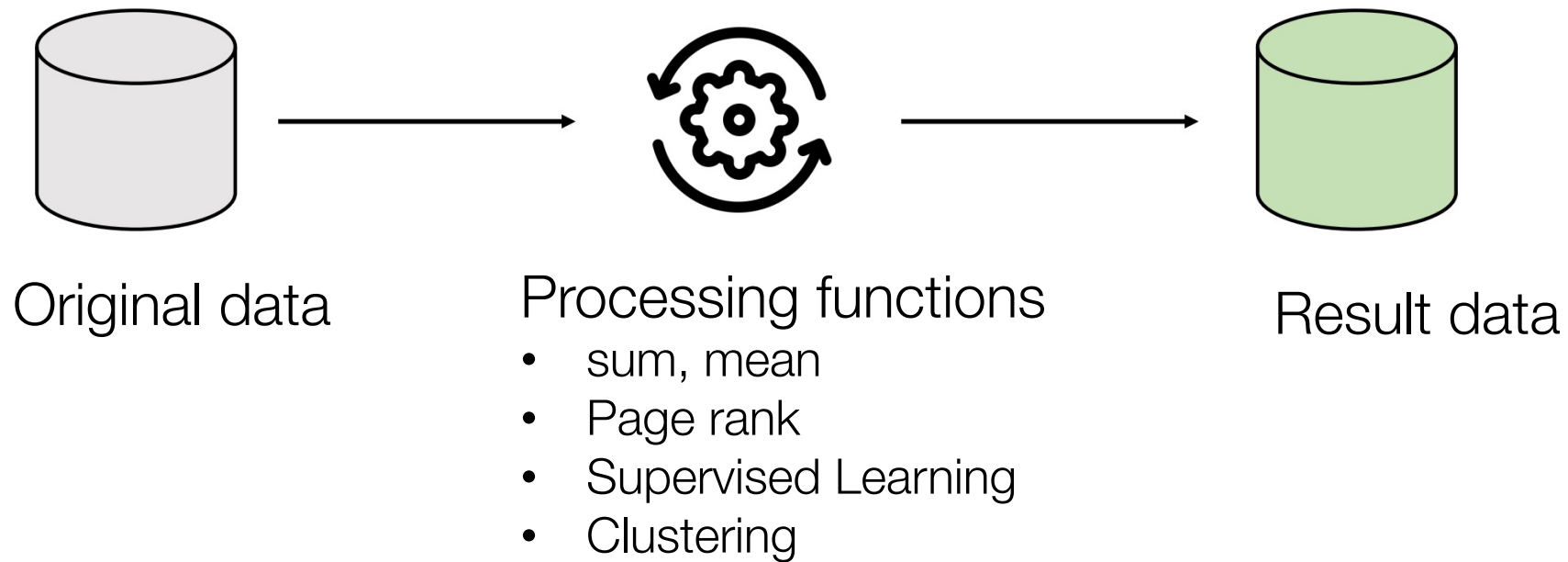


“Divide and Conquer”

Combine



Data processing abstraction



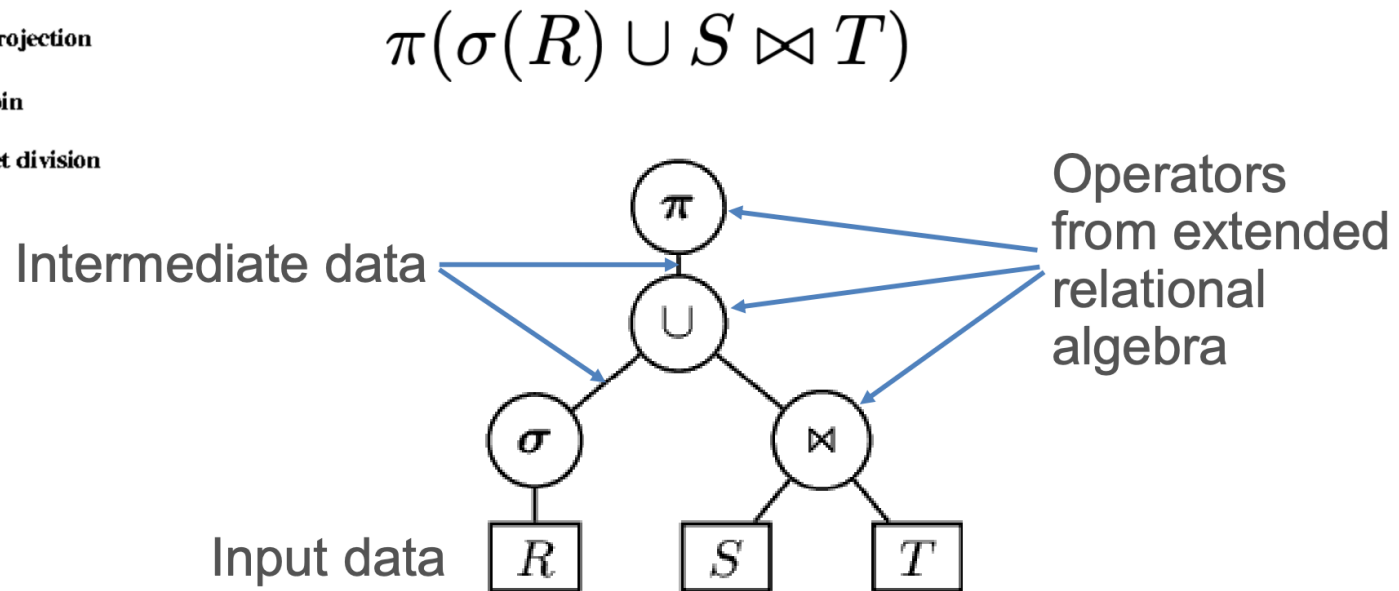
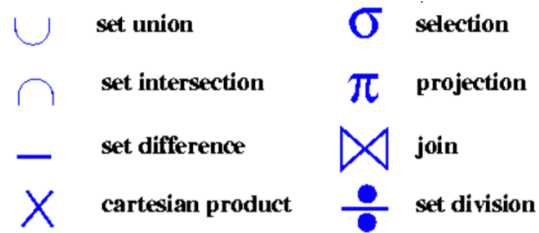
Q: How to represent various processing functions?

How to represent processing functions?

Dataflow Graph: common in parallel data processing

- A directed acyclic graph (DAG) representation of a program
 - **Vertices**: computing operations
 - **Edges**: data flowing directions (hence data dependency)
- Examples
 - Relational dataflows: RDBMS, Pandas
 - Matrix/tensor dataflows: NumPy, PyTorch, TensorFlow
- Enables us to reason about data-intensive programs at a higher level

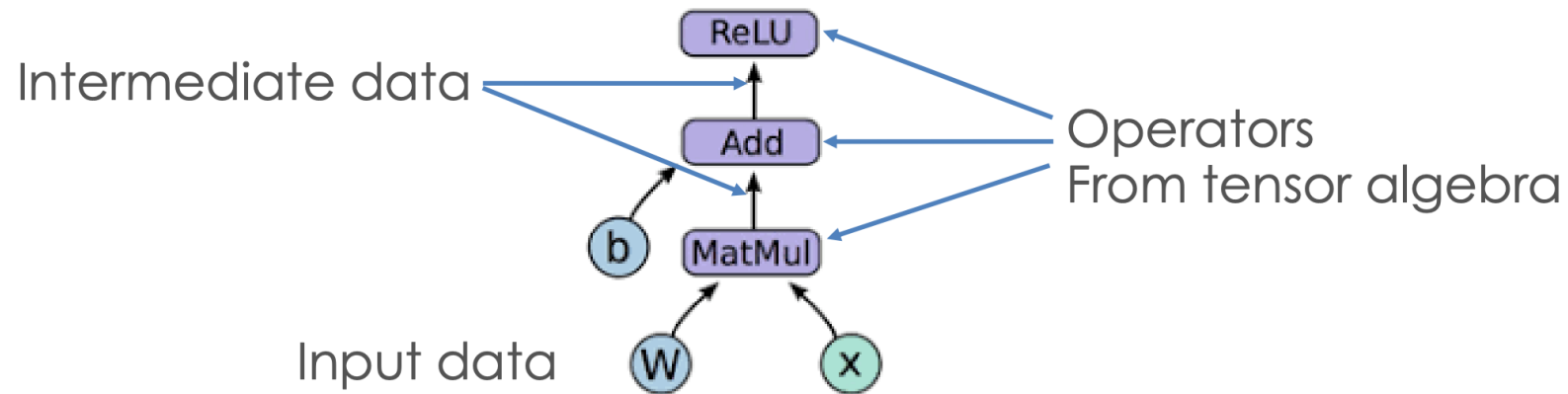
Example: relational dataflow graph



Logical Query Plan in the DB systems world

Example: machine learning dataflow graph

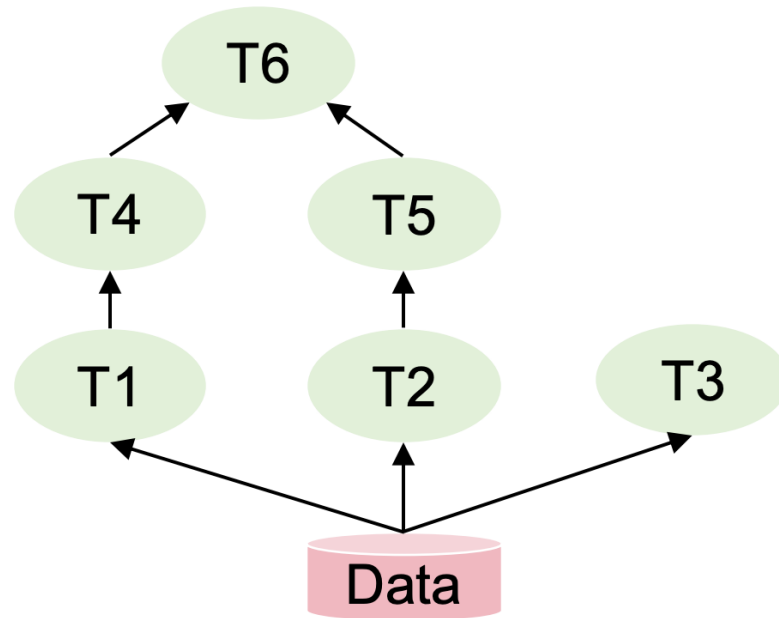
$$\text{ReLU}(W X + b)$$



Neural network computational graph in ML systems

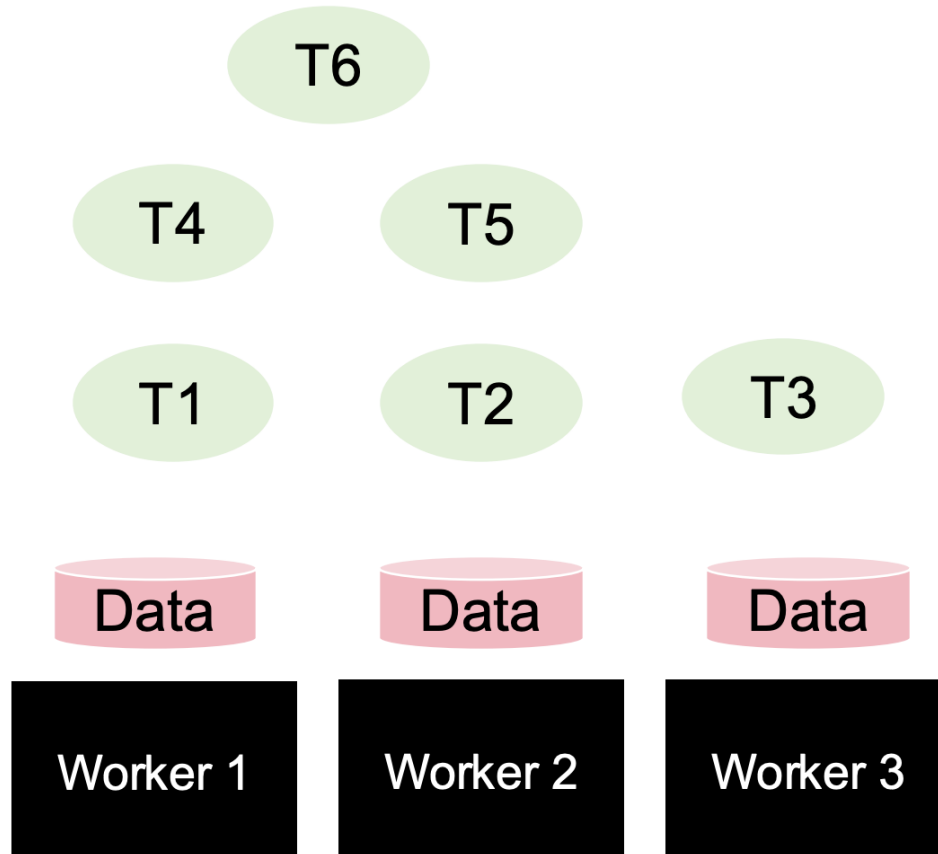
Task parallelism

Split up tasks and execute them across workers



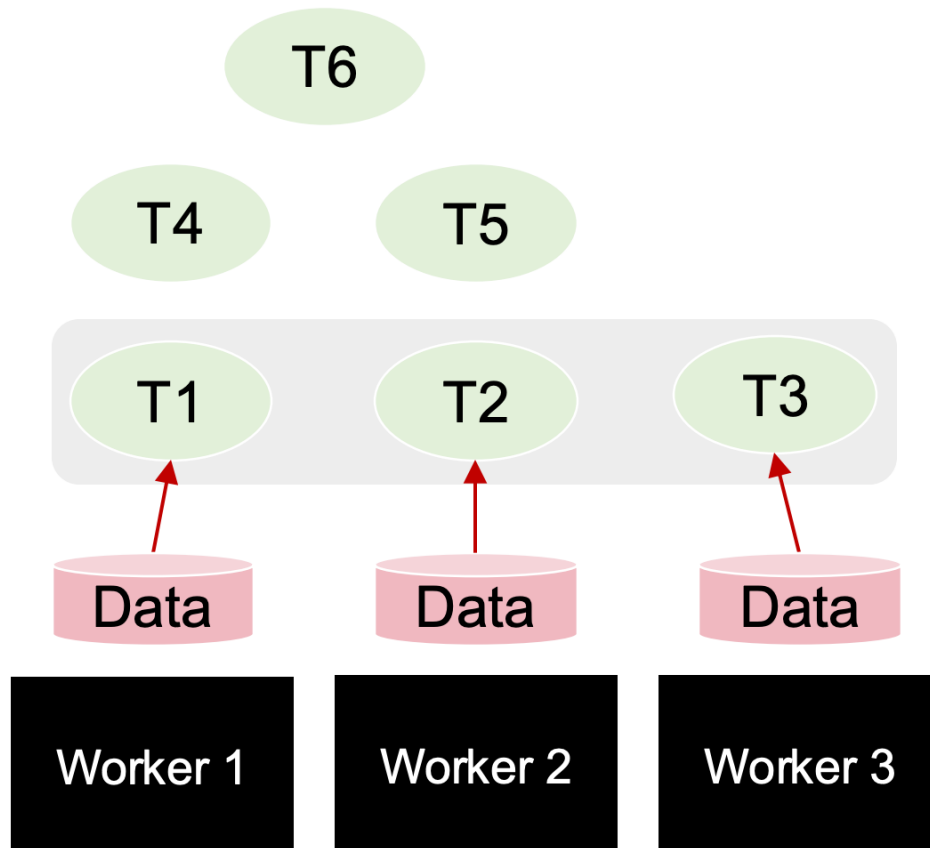
Task DAG

Task parallelism



S1: Copy whole dataset to all workers

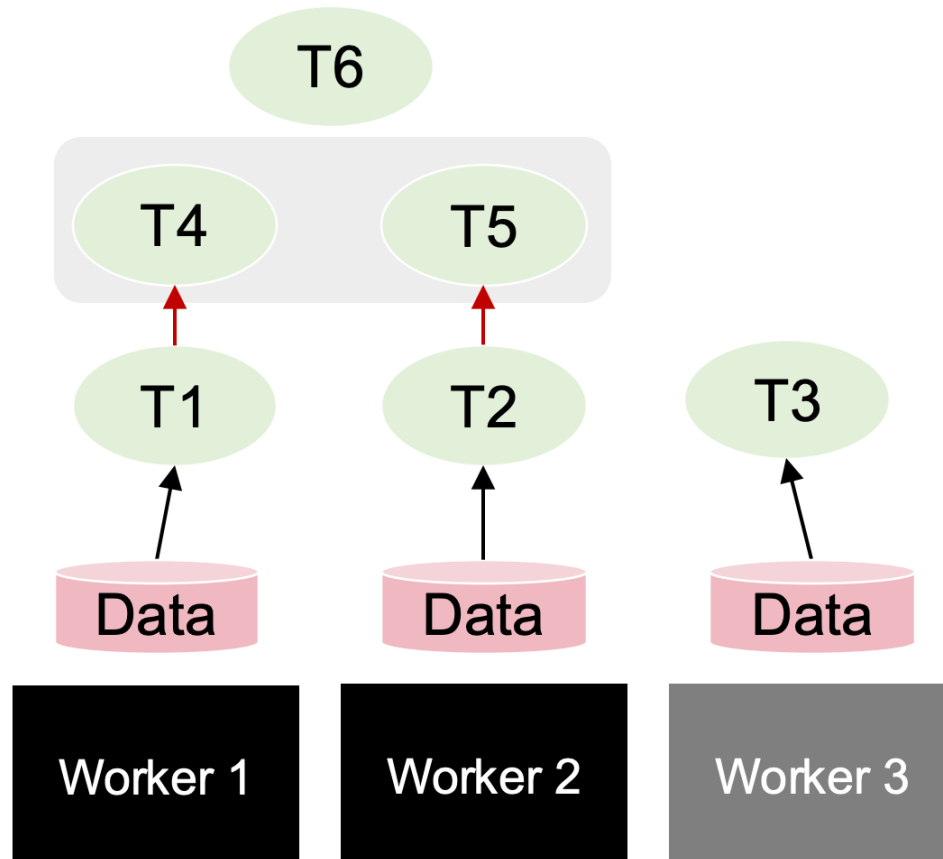
Task parallelism



S2: Schedule T1 to W1, T2 to W2, T3 to W3

S1: Copy whole dataset to all workers

Task parallelism

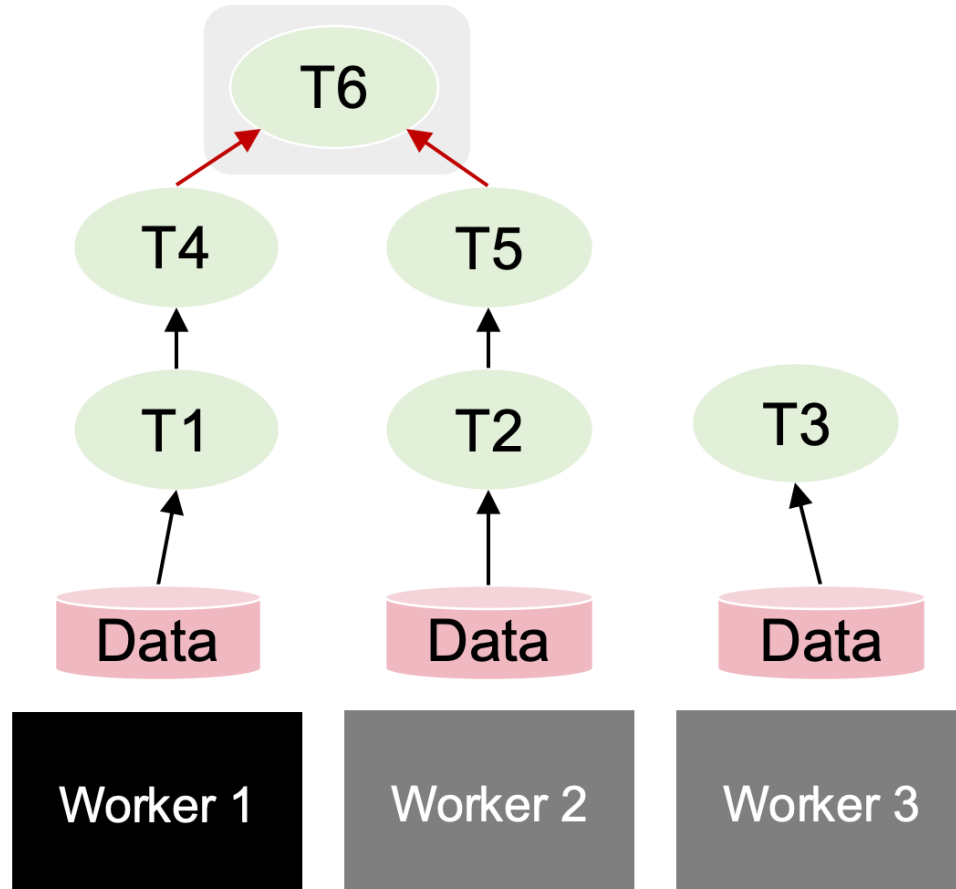


S3: Run T4 after T1 on W1, run T5 after T2 on W2; after T3, W3 is idle

S2: Schedule T1 to W1, T2 to W2, T3 to W3

S1: Copy whole dataset to all workers

Task parallelism



S4: After T4 and T5 ends, run T6 on W1; after T5, W2 is idle

S3: Run T4 after T1 on W1, run T5 after T2 on W2; after T3, W3 is idle

S2: Schedule T1 to W1, T2 to W2, T3 to W3

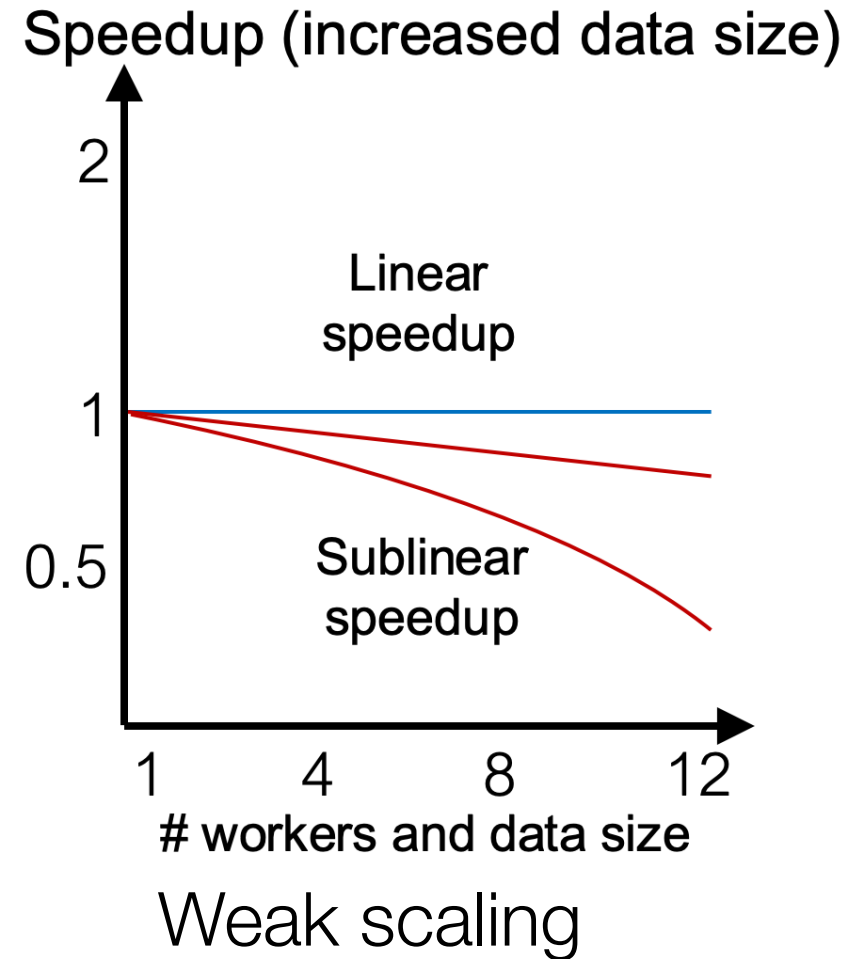
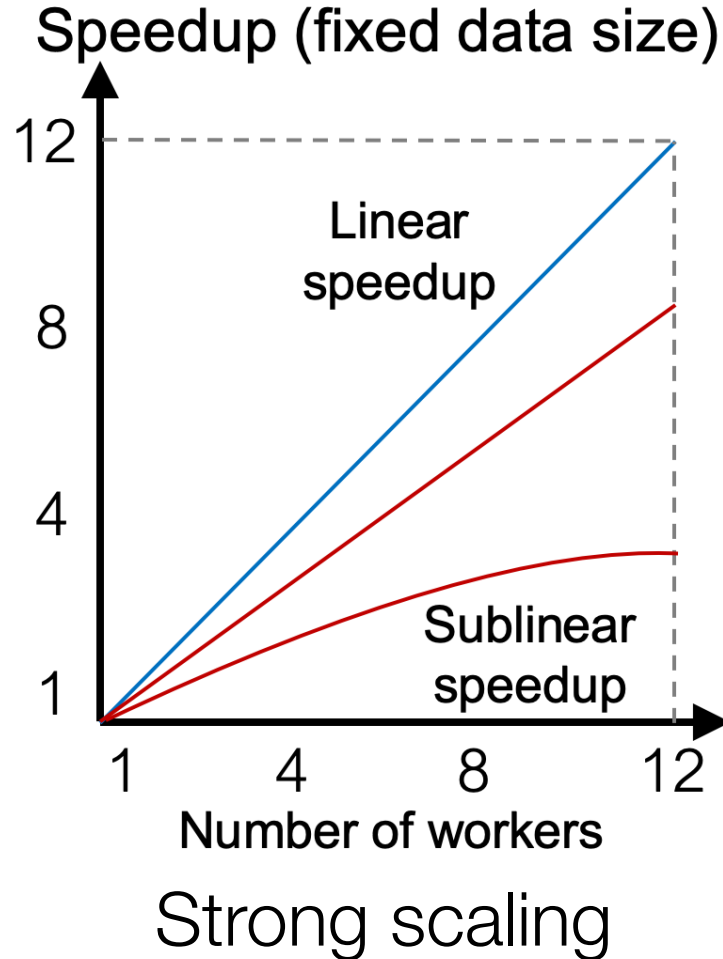
S1: Copy whole dataset to all workers

Quantifying Benefit of Parallelism: Speedup

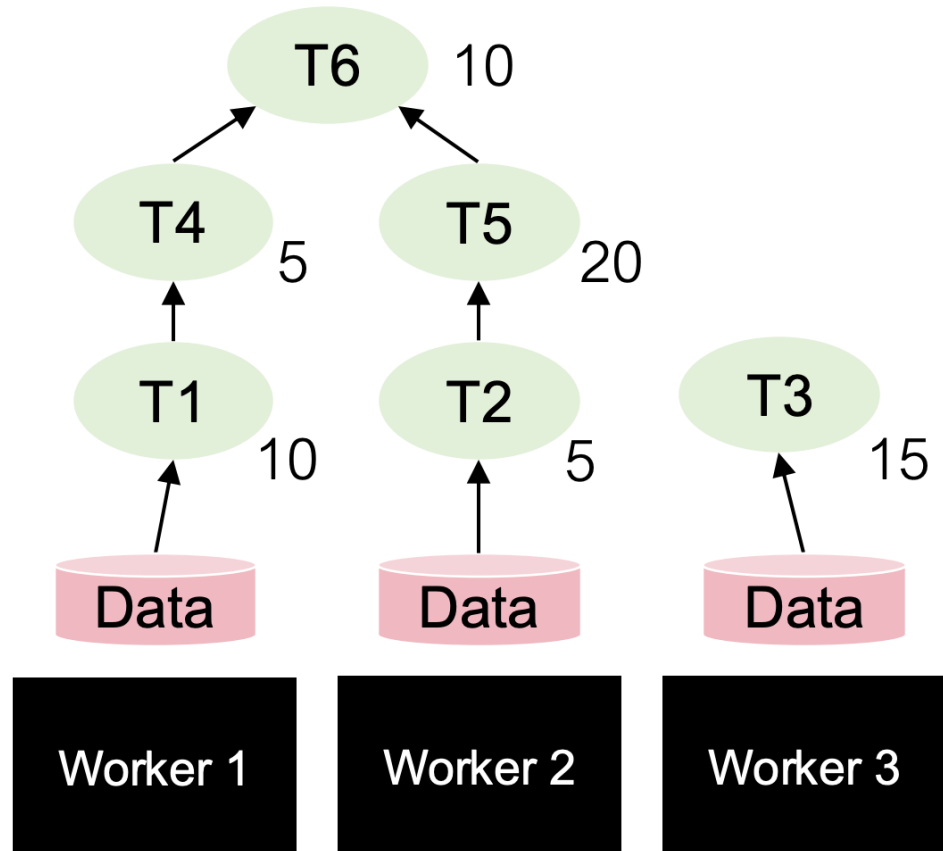
$$\textbf{Speedup} = \frac{\text{Completion time given 1 worker}}{\text{Completion time given } N \text{ worker}}$$

Q: Given n workers, can we get a speedup of n ?

Quantifying speedup



Quantifying speedup



Task completion time varies

Q1: What's the job completion time with 1 worker?

Q2: What's the job completion time with 3 workers?

Q3: What's the speedup?

Speedup implications

Speedup affected by:

- The “parallelizable” portion of a program
- Degree of parallelism
- Execution times of individual tasks
- Synchronization and communication
- ...

Recap: communication



- Establish TCP connection
- Send/Recv data

Point-to-point communication

Communication patterns

Point-to-point communication

- Single sender, single receiver
- Relatively easy to implement efficiently

Collective communication

- Multiple senders and/or receivers
- Complex patterns (broadcast, scatter, gather, ...)

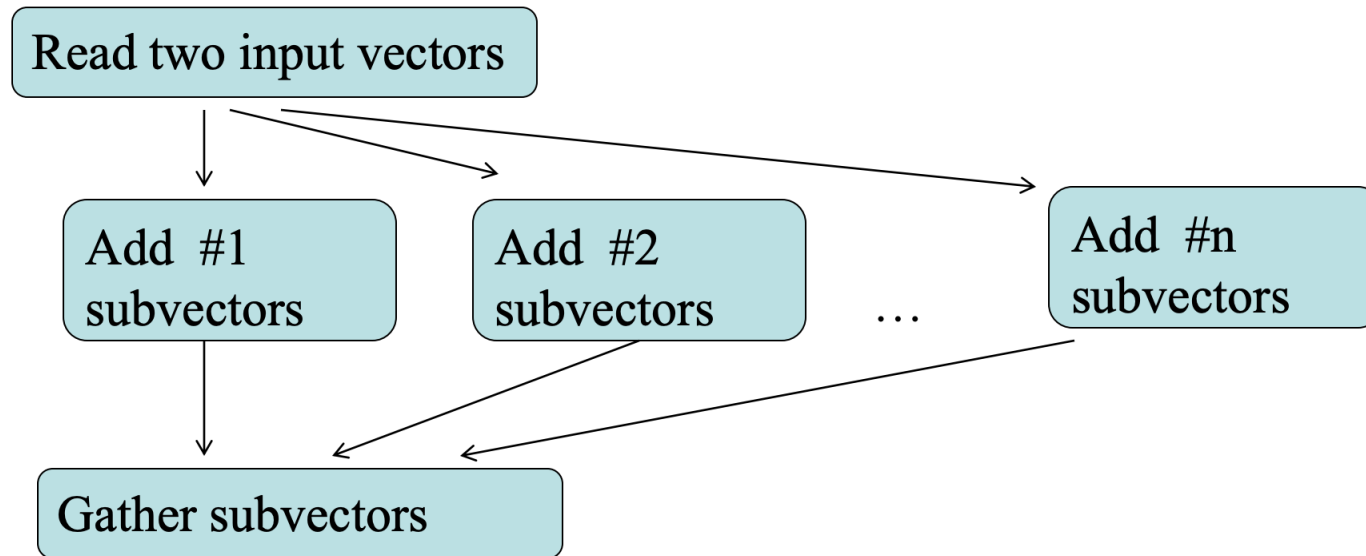
Case study: sum of two long vectors

$$\begin{aligned}\mathbf{x} + \mathbf{y} &= (x_0, x_1, \dots, x_{n-1}) + (y_0, y_1, \dots, y_{n-1}) \\ &= (x_0 + y_0, x_1 + y_1, \dots, x_{n-1} + y_{n-1}) \\ &= (z_0, z_1, \dots, z_{n-1}) \\ &= \mathbf{z}\end{aligned}$$

Sequential version for computing the sum

```
void Vector_sum(double x[], double y[], double z[], int n) {  
    int i;  
  
    for (i = 0; i < n; i++)  
        z[i] = x[i] + y[i];  
} /* Vector_sum */
```

Case study: sum of two long vectors



Parallel implementation

- Divide each vector into n subvectors and distribute
- Add subvectors in parallel
- Gather the sum subvectors

Scatter and gather!

Case study: gradient update in ML

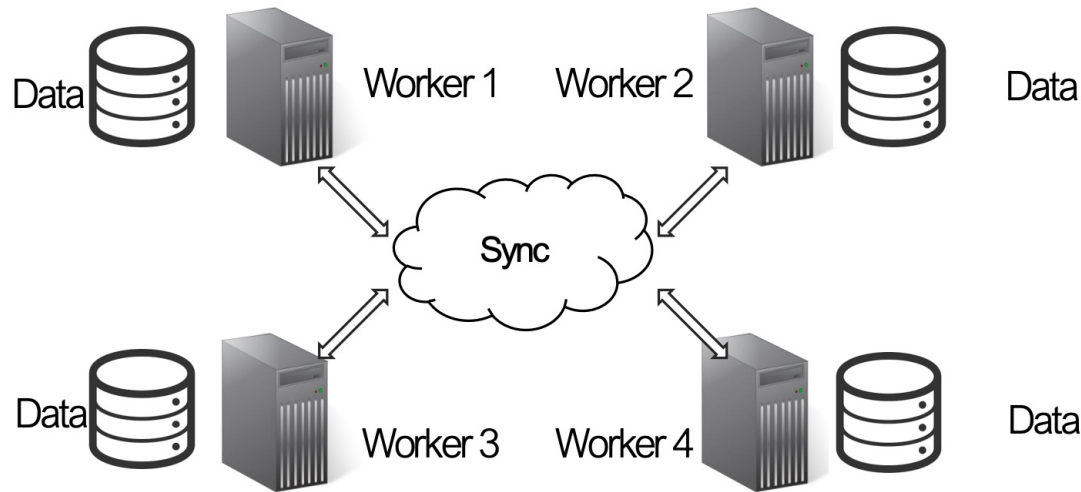
Gradient / backward computation

$$\theta^{(t)} = \theta^{(t-1)} + \boxed{\varepsilon \cdot \nabla_{\mathcal{L}}(\theta^{(t-1)}, D^{(t)})}$$

↑ ↑
objective data

Case study: gradient update in ML

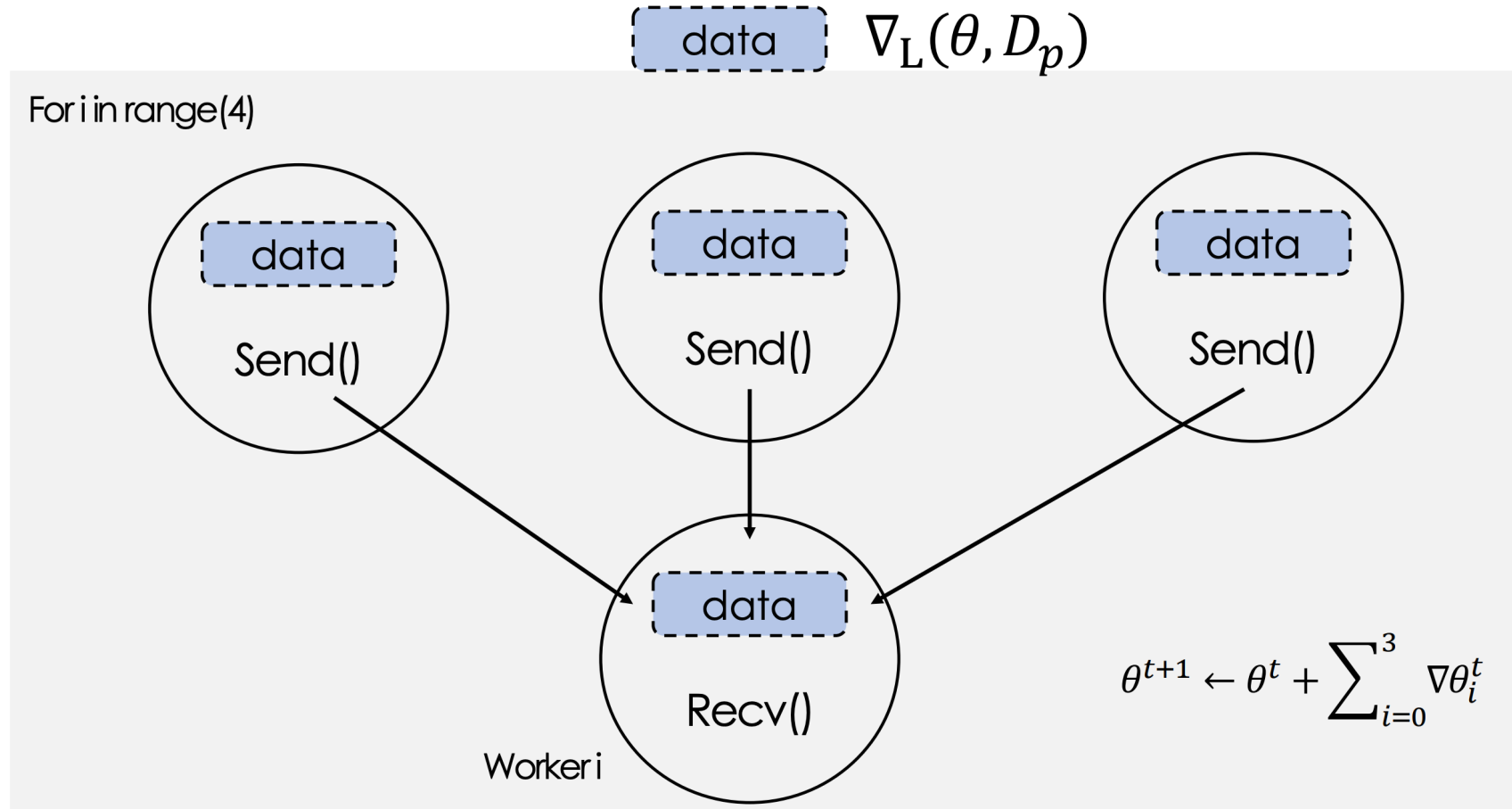
Large amount data distributed across workers



$$\theta^{(t+1)} = \theta^{(t)} + \varepsilon \sum_{p=1}^P \nabla_{\mathcal{L}}(\theta^{(t)}, D_p^{(t)})$$

How to perform the sum?

Case study: gradient update in ML



All-Reduce!

Why collective communication

Programming Convenience

- Use a set of well-defined communication primitives to express complex communication patterns

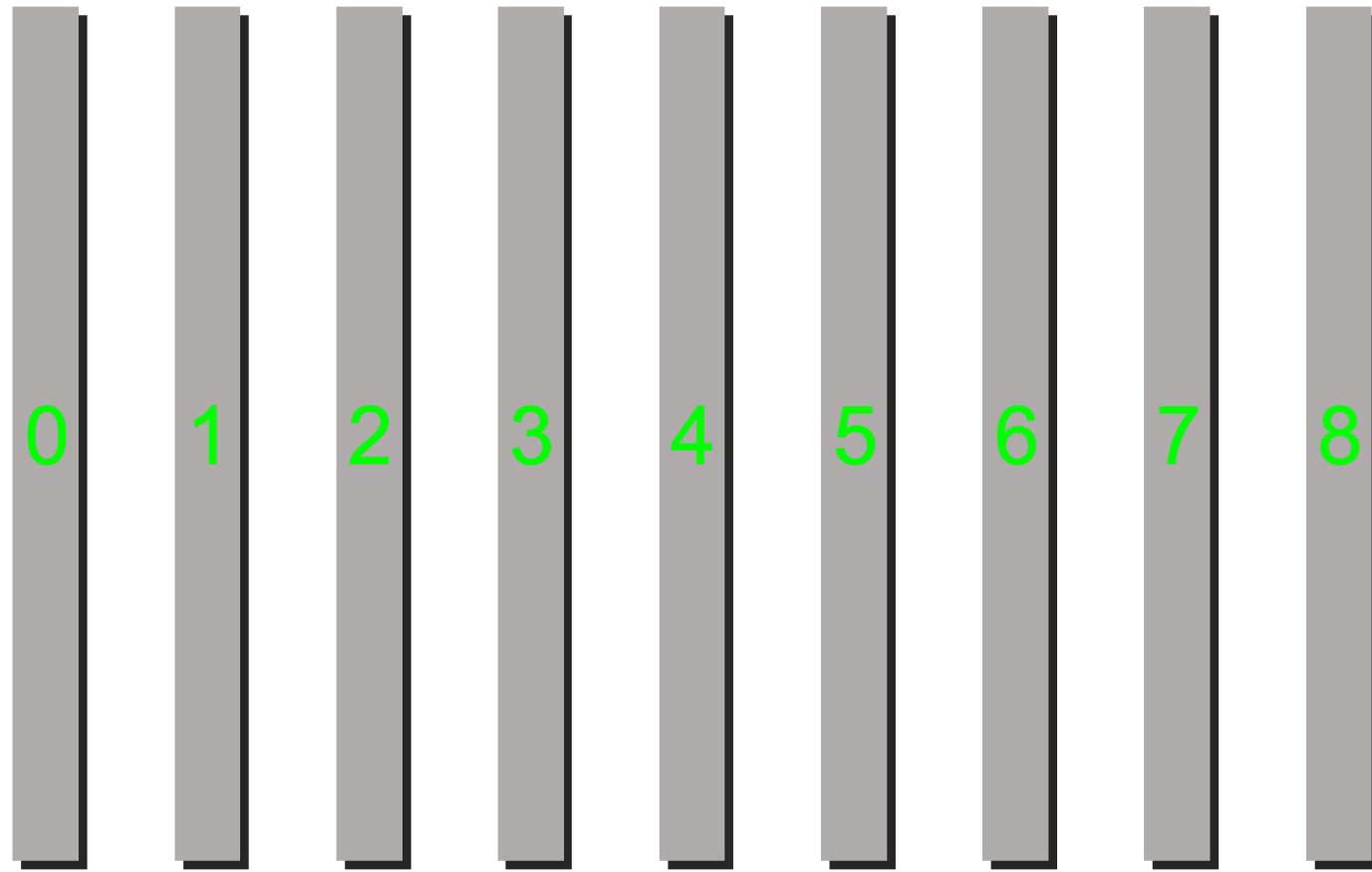
Performance

- Since they are well defined and well structured, we can optimize them to the extreme

Many applications

- Scientific computing, HPC, big data processing, ML

Make it formal



A 1D Mesh of workers (or devices, or nodes)

Model

- a node can send directly to any other node (maybe not true)
- a node can simultaneously receive and send
- cost of communication
 - sending a message of length n between any two

$$\alpha + n \beta$$

- if a message encounters a link that simultaneously accommodates M messages, the cost becomes

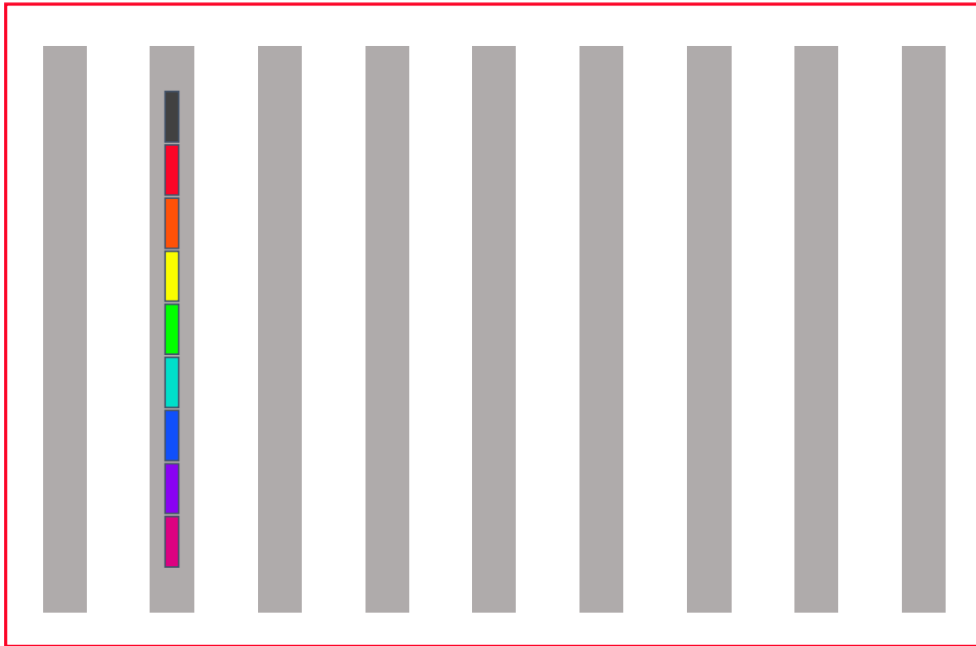
$$\alpha + Mn \beta$$

Collective communication primitives

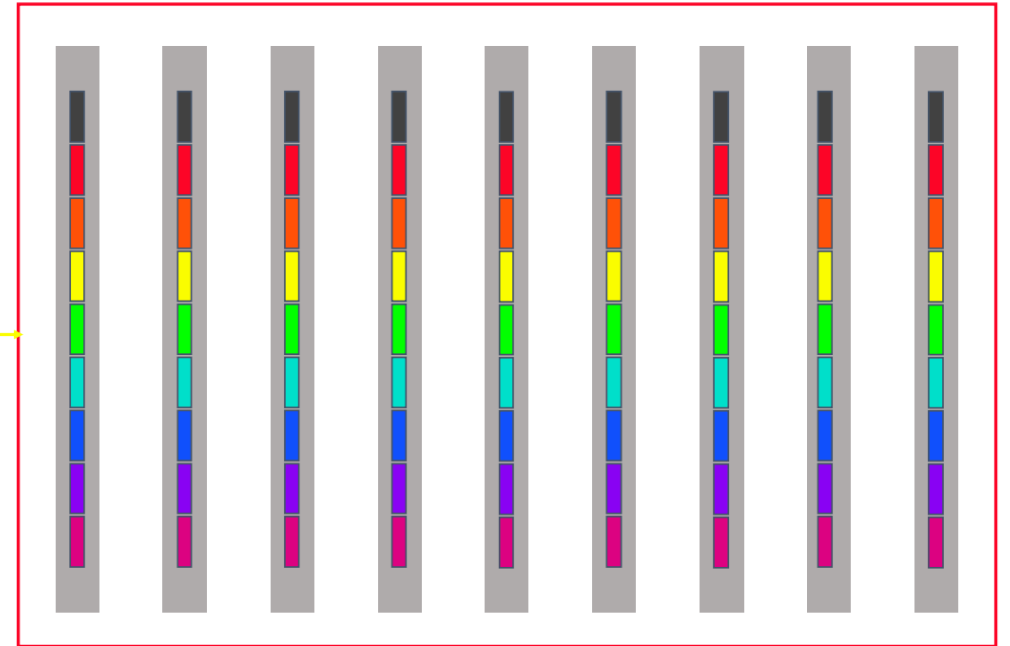
- Broadcast
- Reduce(-to-one)
- Scatter
- Gather
- Allgather
- Reduce-scatter
- Allreduce

Broadcast

Before

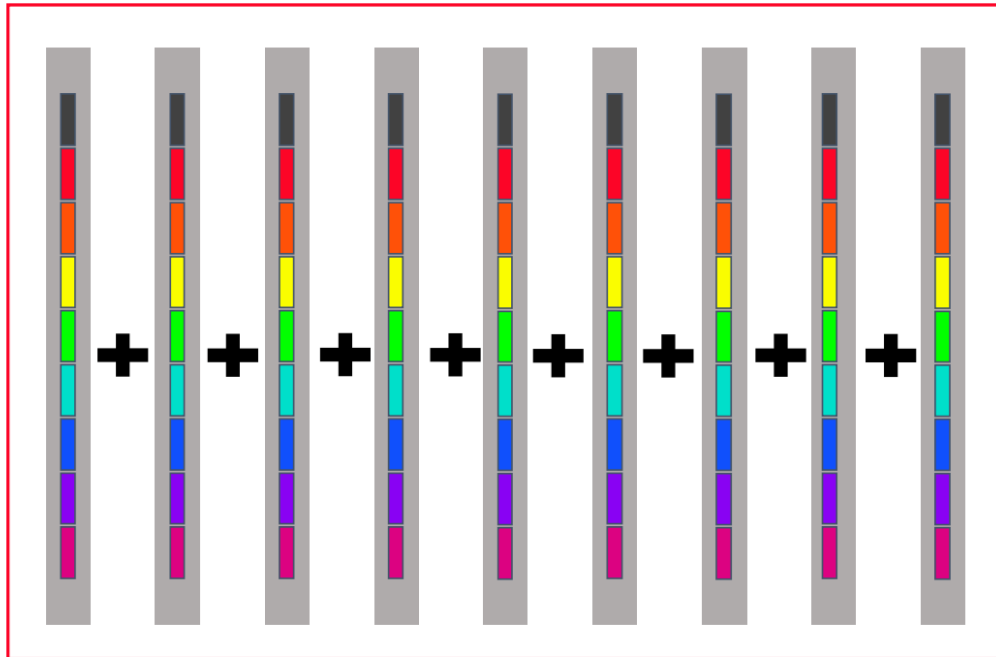


After



Reduce

Before

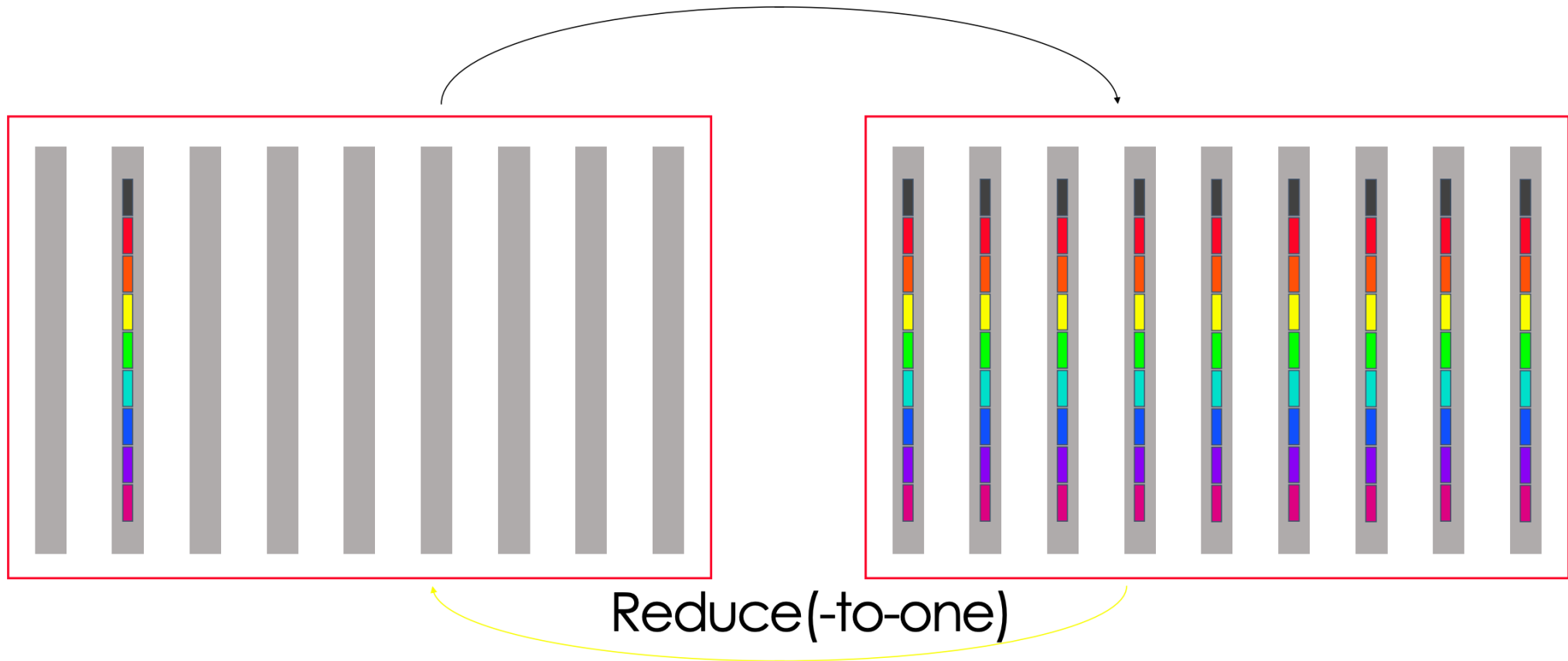


After



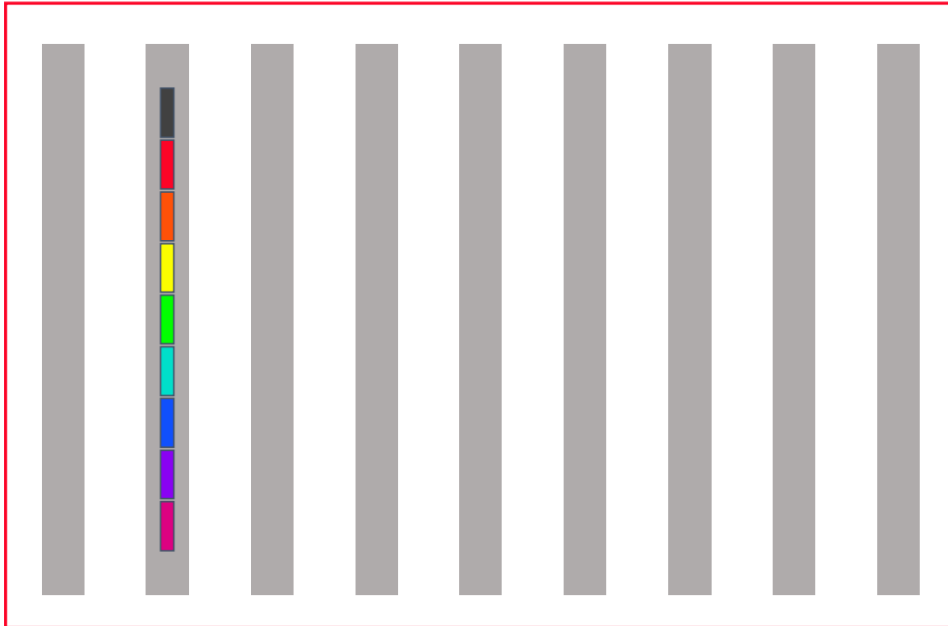
Broadcast/Reduce

Broadcast

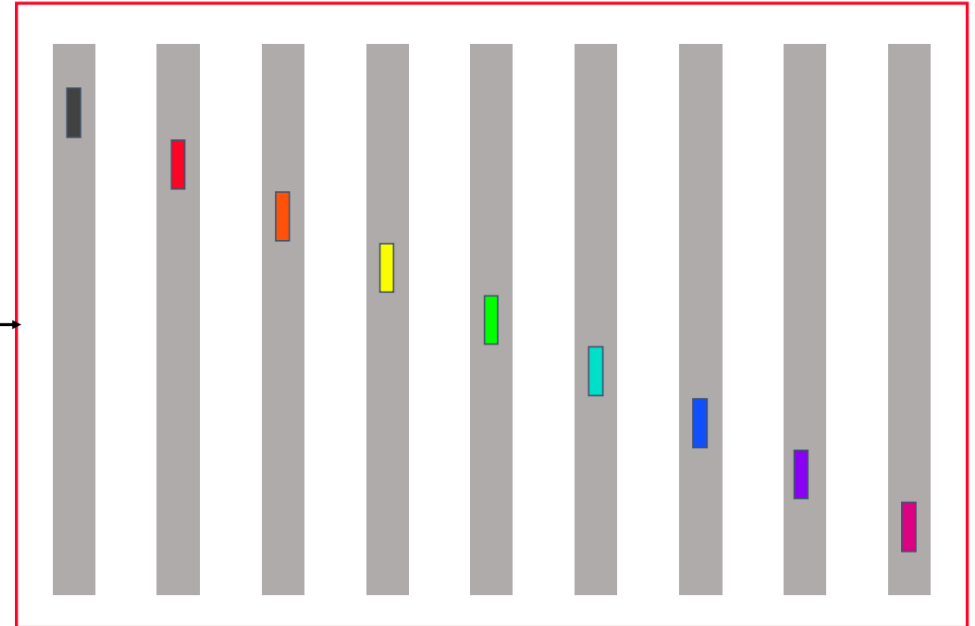


Scatter

Before

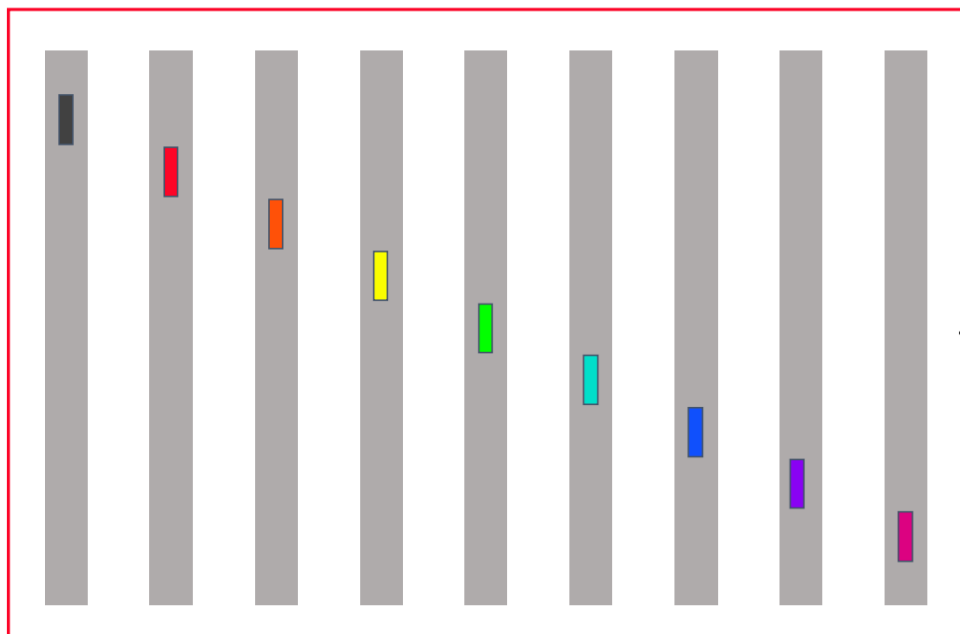


After

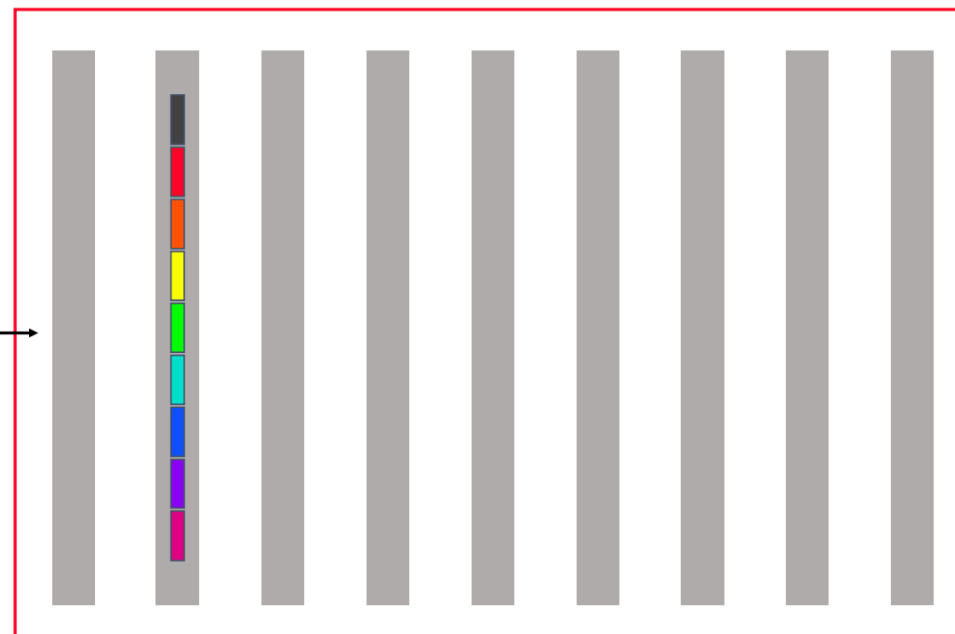


Gather

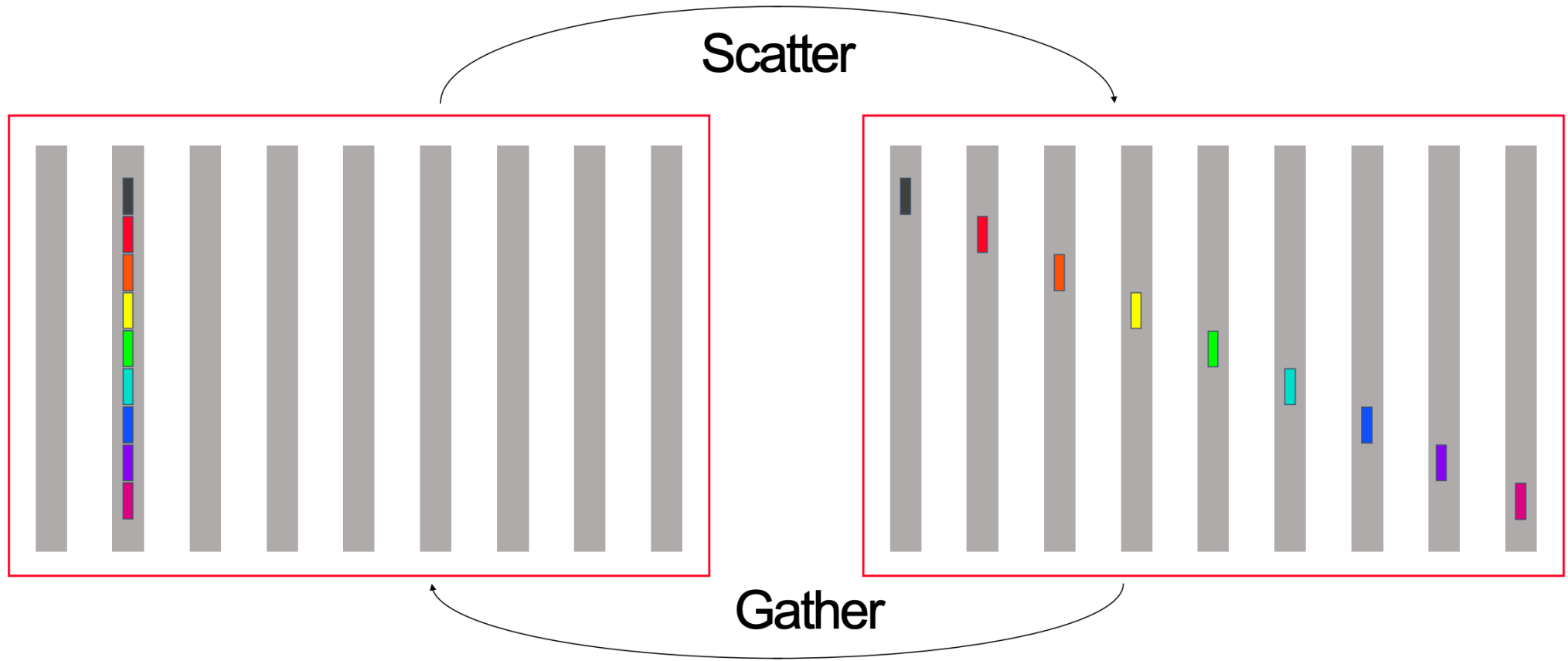
Before



After

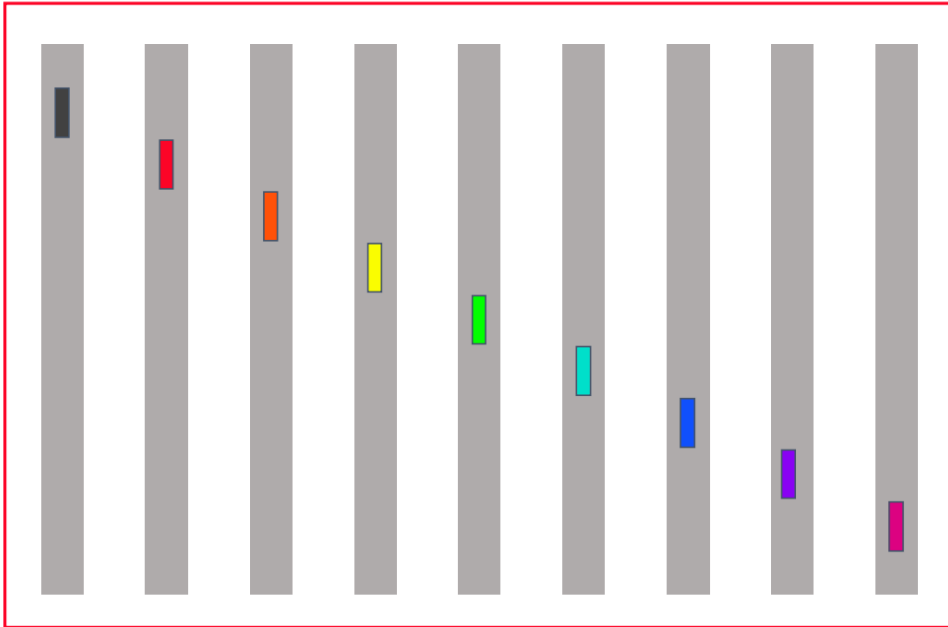


Scatter/Gather

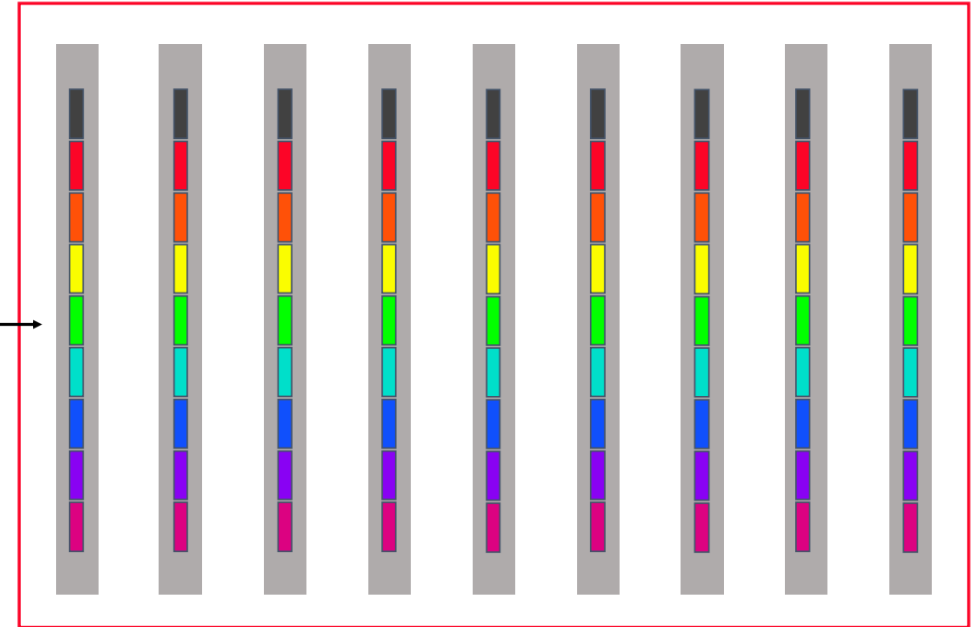


Allgather

Before

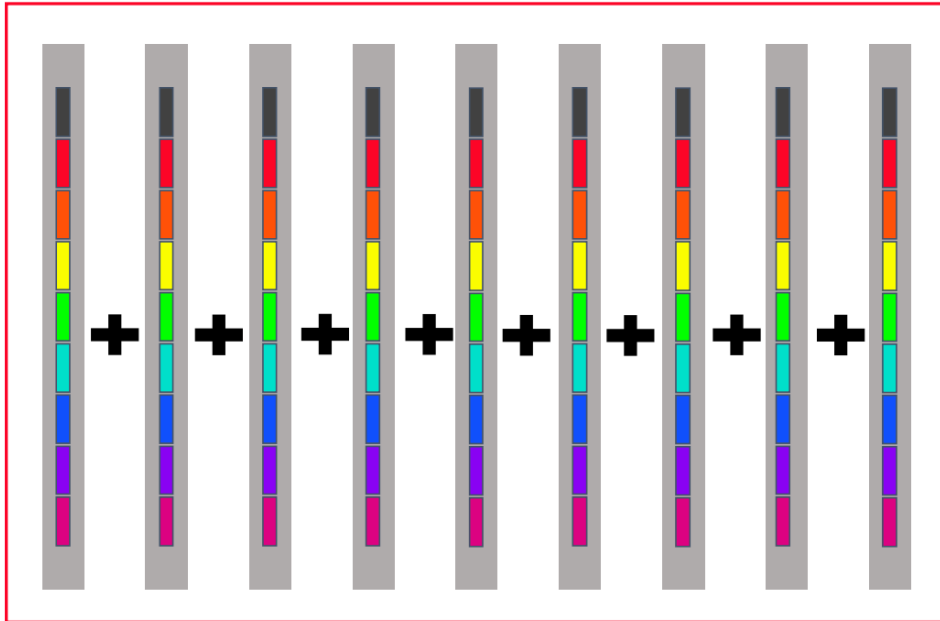


After

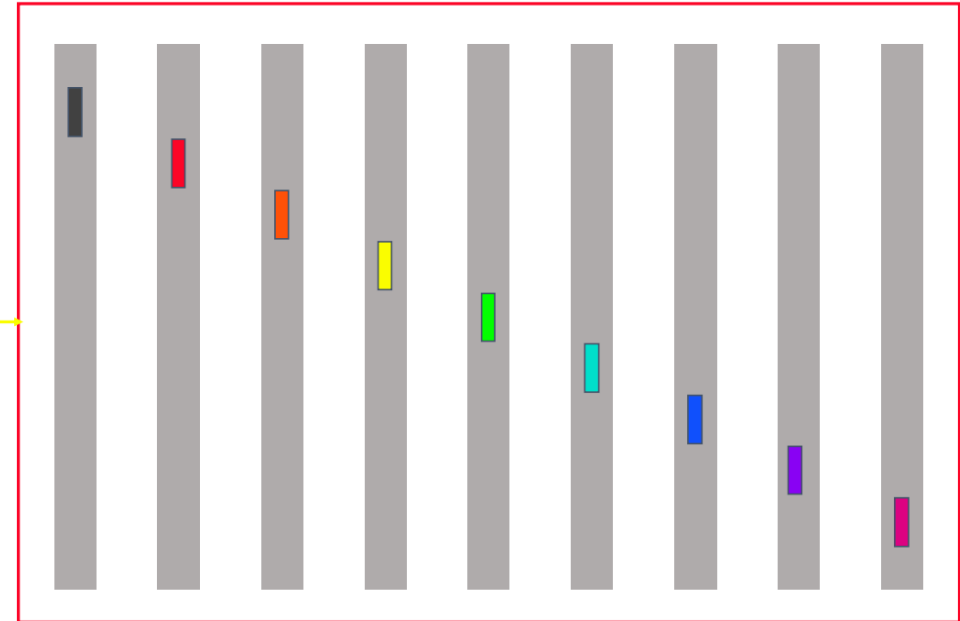


Reduce-scatter

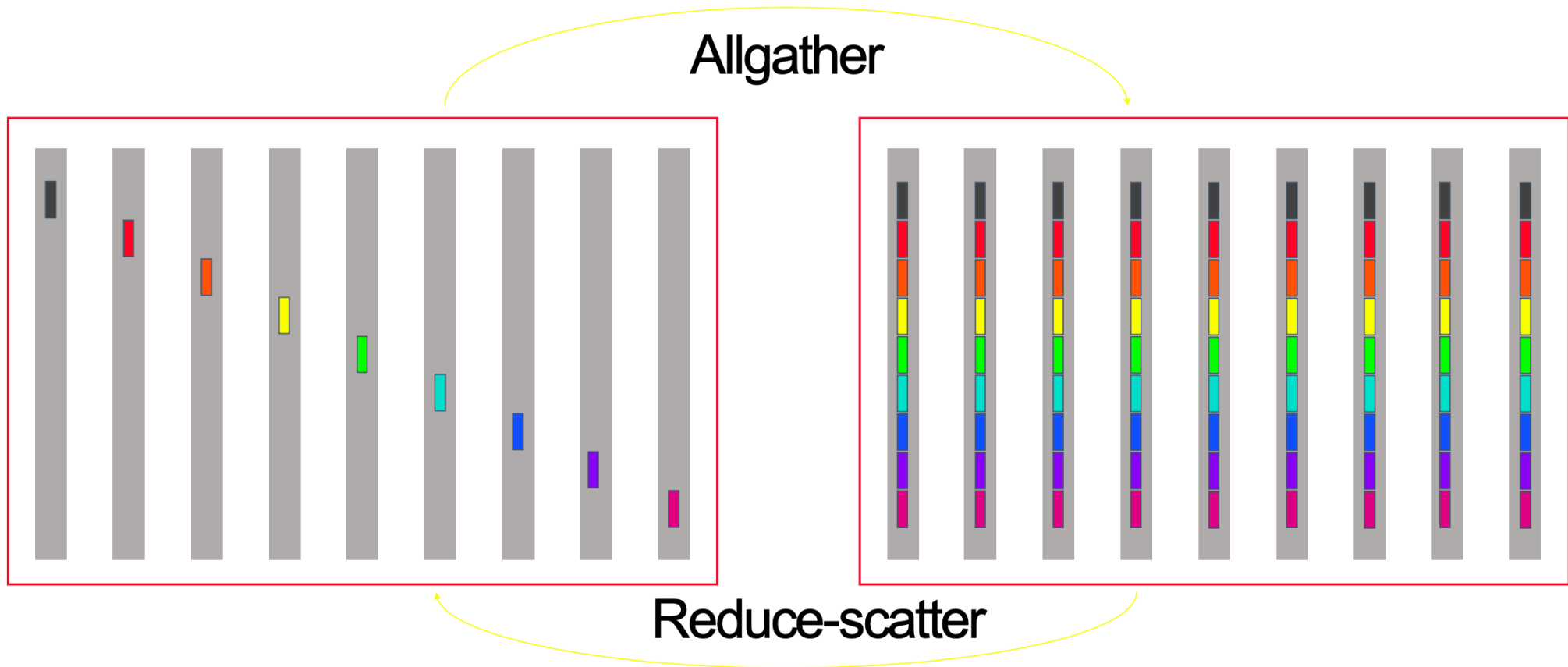
Before



After

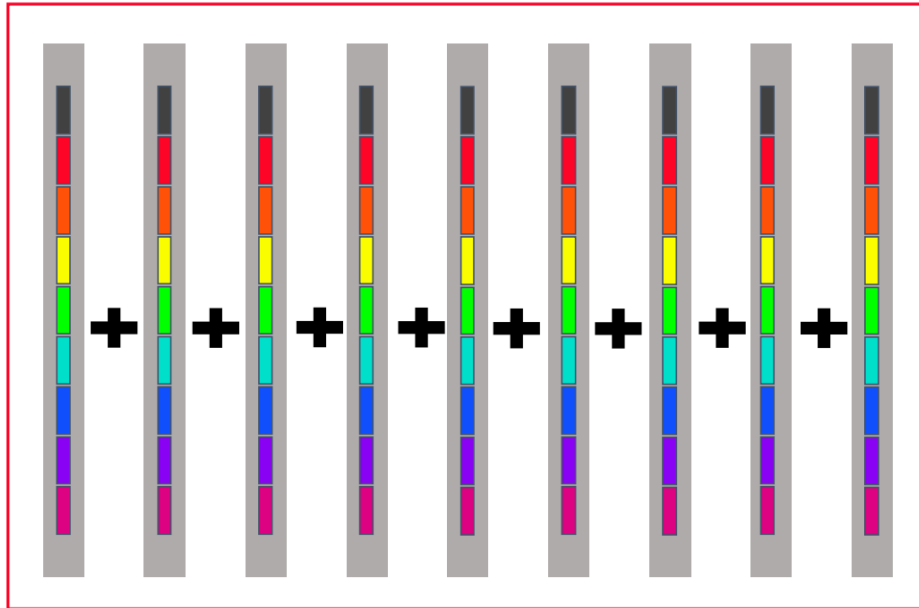


Allgather/Reduce-scatter

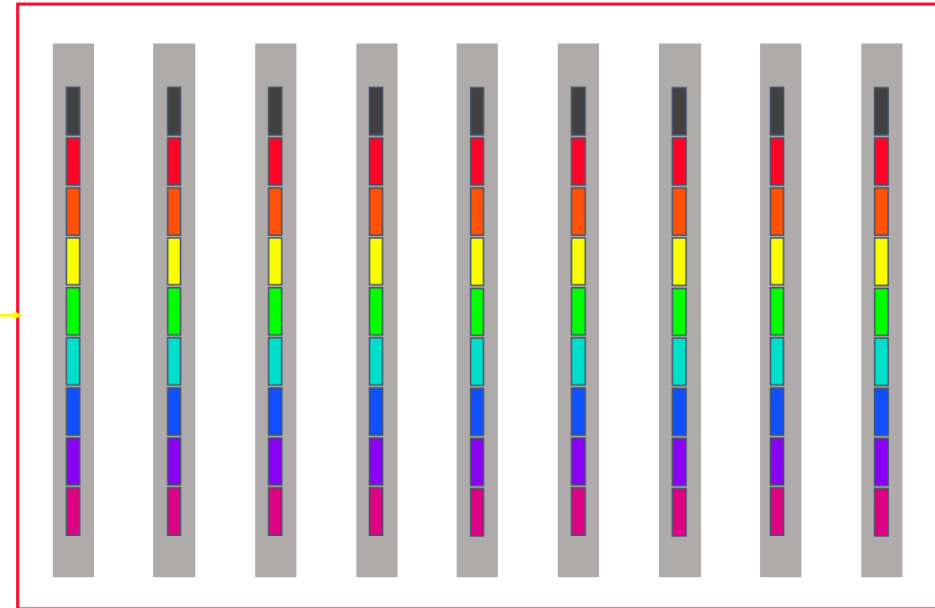


Allreduce

Before



After



Reference & Credits

- John Wiley & Sons, “Collective communication: theory, practice, and experience”
- Some slides adapted from course slides of DS5110 in UVA
- Some slides adapted from course slides of DSC204a in UCSD